MAGENTRA: An Entropy-Aware Harness for Agentic Coding

Muhammet Ali Öztürk
Hacettepe University, Computer Engineering (Kalai Labs)
muhammet_ozturk@hacettepe.edu.tr

August 2026

Abstract

Agentic coding tools now write a substantial share of new code, and with that share comes a familiar risk in a new form: software entropy. The tendency of codebases to accumulate duplication, drift, and unverified change has been described in the software-engineering literature for decades; a code-writing agent that acts quickly and confidently can accelerate all three. This paper describes MAGENTRA, an open-source agentic coding harness built around a provider-agnostic, frontend-agnostic engine — usable through a desktop workbench, a terminal interface, or headless — whose design treats entropy control as a harness responsibility rather than a hoped-for model behaviour. Because every mechanism is open and editable, individuals and organisations can adopt it without the privacy and security doubts a closed agent raises. We describe the mechanisms this position produces: a clarification step that asks the user shape-defining questions before open-ended work starts; a multi-language import graph the agent queries for impact analysis before editing; a search-before-write reuse gate that surfaces existing code when the agent is about to write a near-duplicate; end-of-turn "finishing checks" that ask for runtime evidence, flag self-authored test doubles as circular evidence, and run a final self-verification round; and a permission engine organised around reversibility, whose fully autonomous mode is balanced by an automatic pre-turn snapshot of the working tree. A harness can only reduce entropy, not eliminate it; we present these mechanisms as a partial mitigation, with fuller answers the subject of ongoing research at Kalai Labs. The paper also describes a benchmark adapter that runs the unmodified engine on Terminal-Bench, and outlines a roadmap toward trainable, shareable role agents whose accumulated experience is portable across repositories. Evaluation is ongoing; this paper focuses on the system and the design rationale connecting it to long-standing software-engineering principles.

1 Introduction

Large language models can now carry out multi-step software tasks with considerable autonomy: reading a repository, editing files, running commands, and iterating on failures. A growing family of tools — command-line agents, IDE integrations, and autonomous software engineers — packages this capability for daily use. The component that turns a conversational model into a working coding agent is commonly called the *harness*: the system prompts, tools, permission rules, memory, and control loop that surround the model and shape every action it takes. Surveys of the area note that the field's centre of gravity has shifted from algorithmic novelty toward exactly these engineering concerns — "system reliability, process management, and tool integration" [1]; this paper is about one harness built around one such concern.

This paper starts from an observation that predates language models entirely. Software systems decay unless energy is spent maintaining them: requirements shift, duplicated logic diverges, documentation rots, and complexity accumulates. Lehman formulated this as laws of software evolution [2]; Hunt and Thomas popularised the term *software entropy* and the "broken windows" framing — one tolerated flaw invites the next [3–5]; Parnas described the same phenomenon as software aging [6]; and Cunningham's debt metaphor gave it an economic reading [7]. None of these authors were writing about AI. Their subject was the human maintainer, whose attention is finite and whose incentives favour the quick change.

An agentic coding tool sharpens this old problem in three ways. First, *volume*: an agent produces edits faster than a human reviews them, so any per-edit tendency toward duplication compounds quickly. Industry analyses of AI-assisted repositories report measurable rises in duplicated and churned code [8, 9]. Second, *locality*: a model sees the context it is given, not the repository as a whole; the function it is about to write may already exist two directories away, invisible unless the harness makes it visible. Third, *verification asymmetry*: a model that wrote a change is disposed to believe the change works; without an external demand for runtime evidence, "it compiles" or "the test I also wrote passes" stand in for observation. These failure modes have been named repeatedly by practitioners building and studying agents [10, 11].

Magentra is an open-source agentic coding system: a provider-agnostic engine that runs behind any frontend — today a desktop workbench, a terminal interface, a headless server host, and a benchmark driver — built around the position that these failure modes are the *harness's* job to counter. The discipline lives in mechanisms the user can inspect and edit, and that openness is itself a design goal: an organisation adopting an agentic harness should be able to read every instruction the model receives, keep every byte of state on its own machines, point the engine at endpoints it trusts, and change any behaviour it disagrees with — adoption without privacy or security doubts. Closed tools may implement similar checks internally; here they are open and easily changeable. Concretely, the harness:

- assembles its system prompt from a registry of named, individually editable sections, several of which encode maintenance discipline drawn from the software-engineering canon (reuse before writing, replace in place, fix the mechanism rather than the instance) (§5.1);
- builds a multi-language import graph of the workspace and exposes it to the agent as a query tool — ranked context slices, dependency closures, and "blast radius" impact sets — so structural knowledge is one cheap call away instead of many speculative file reads (§5.2);
- intercepts the creation of new source files with a *reuse gate* that checks, without any model call, whether similar code already exists and whether the agent searched for it first, and if not, names the closest matches (§5.3);
- ends each working turn with *finishing checks*: a runtime-evidence reminder when code changed but nothing was executed, a circular-evidence warning when the only evidence came from test doubles the agent wrote itself, and a final self-verification round that must either resume work or conclude the turn (§5.5);
- scopes autonomy with a permission engine organised around reversibility, and precedes open-ended work with a clarification step that asks the user decision-changing questions before acting (§5.4).

We use *entropy-aware* for this design position: the harness assumes entropy pressure by default and pays a small, bounded cost every turn — extra prompt text, an index lookup,

sometimes an extra inference round — to push against it. The mechanisms are deliberately advisory where they can be and blocking only where damage is irreversible; a wrongly blocked action teaches the model to route around the safeguard, while a well-timed reminder leaves the decision, and the responsibility, with the agent (§5.3). We are equally explicit about the ceiling: a harness reduces entropy, it does not eliminate it. What is presented here is a partial mitigation; fuller answers to entropy in agentic and "vibe" coding are an ongoing research direction at Kalai Labs.

This paper focuses on the system and its rationale; evaluation is ongoing, with the learning components of §7 under test at the time of writing and benchmark evaluation (§6) ahead. What the paper offers is (i) an account of a working harness whose mechanisms are traceable to named entropy problems, (ii) the design rationale behind each mechanism, and (iii) a roadmap for portable, community-trainable role agents built on the same substrate. The source code is public.[1]

2 Background and Motivation

2.1 Software entropy before AI

The claim that software decays by default is one of the oldest empirical generalisations in software engineering. Lehman's laws of software evolution — continuing change, increasing complexity, declining quality absent explicit work against it — were drawn from longitudinal observation of industrial systems [2, 12]. Brooks located the essential difficulty of software in its invisible, arbitrarily complex structure [13, 14]. Parnas's "software aging" described how a program's structure degrades as changes are made by people who do not fully understand its design [6] — a description that transfers uncomfortably well to a language model editing an unfamiliar repository. Perry and Wolf's foundational treatment of software architecture supplies the standard vocabulary for the decay modes: *erosion*, from violating architectural principles, and *drift*, from insensitivity to the architecture — divergence between intended and implemented [15]; an agent that edits without structural knowledge is a drift generator in exactly this sense. Nor is entropy only a metaphor in this literature: Hassan showed that the Shannon entropy of a project's code-change process predicts faults better than well-known historical predictors such as prior modifications or prior faults [16] — disorder in how a codebase changes is a measured, validated risk signal. The practitioner literature converted these observations into working discipline: Hunt and Thomas's "don't live with broken windows" and the DRY principle [4], Fowler's refactoring as continuous repair [17], Ousterhout's account of complexity as incremental accumulation of dependencies and obscurity [18], McConnell's treatment of complexity management as the primary technical imperative [19], and Martin's coding discipline norms [20]. MAGENTRA's design takes these books seriously as requirements documents: where the canon says "a maintainer should do X before changing code," the harness asks what mechanism would make an agent actually do X (§5).

2.2 Agentic coding and its named failure modes

The shift toward software whose behaviour is shaped by optimisation and generation rather than line-by-line authorship — anticipated in Karpathy's Software 2.0 essay [21] — leaves humans supervising code they did not write, and practitioners building agentic systems have converged

[1] `https://github.com/kalai-labs/MAGENTRA`

on a set of recurring harness problems, discussed extensively in talks, engineering blogs, and a small but growing literature. A shared observation runs through these accounts: the damage that costs the most — degraded maintainability — is also the damage whose natural feedback arrives slowest, which argues for a harness that raises the signal early, at write time and turn end, rather than waiting for decay to surface [22]. We summarise the problems MAGENTRA explicitly targets; §3 covers the systems landscape.

Context is finite and degrades. Model accuracy on multi-document question answering follows a U-shaped curve in the position of the relevant information — highest at the beginning and end of the input, degraded in the middle, even for explicitly long-context models [23]. A study of 18 models across four providers broadens the point: performance degrades non-uniformly as input grows, well below nominal window limits, degradation appears even on trivial copy tasks, and — most usefully for harness design — focused inputs of a few hundred relevant tokens outperform full ~113k-token contexts across every model family tested [24]. The engineering response is context curation, which practitioners have converged on calling context engineering: treating attention as a budget and filling the window deliberately with "the smallest set of high-signal tokens" the next step needs [25, 26]. MAGENTRA's import-graph slices (§5.2), on-demand addons (§5.7), and structured compaction (§5.6) are context-curation mechanisms: ranked structure instead of wholesale file dumps, procedure bodies loaded only when invoked, history summarised into exactly the decision-preserving form the focused-input results favour.

Generated code accumulates measurable entropy. The evidence now spans several methodologies, and it is consistent. A large-scale academic study of 302,600 verified AI-authored commits across 6,299 repositories found that more than 15% of commits from every assistant studied introduce at least one issue, that 22.7% of tracked AI-introduced issues survive to the latest repository version — and that code smells, not correctness or security defects, constitute 89.3% of all issues found: the dominant harm of AI code in the wild is maintainability [27]. GitClear's longitudinal industry analyses — vendor reports, offered with that caveat — concur: across 211 million changed lines (2020–2024), copy-pasted lines rose from 8.3% to 12.3% while refactoring-indicative "moved" lines fell from 24.1% to 9.5%, with 2024 the first year on record in which copy-paste exceeded moved code [8, 9]; the 2026 follow-up over 623 million changes reports duplicated blocks up 81% since 2023, moved lines down 70%, and — notably — error-*masking* constructs up 47% [28]. Controlled comparisons agree: LLM-generated Java solutions carry on average 63% more code smells than professional reference solutions, and the gap widens with task complexity [29]. The consequence is not hypothetical. Long before AI, Juergens et al. showed the causal chain from duplication to faults: in industrial systems, 52% of code clones were changed inconsistently, yielding 107 developer-confirmed faults [30] — agents now produce clones at rates that compound those base rates. And the entropy taxes agents themselves: controlled experiments find agents up to 13.1% *less* effective at resolving tasks on previously agent-generated code than on human code, a gap that classic maintainability metrics fail to explain [31]. This is entropy in Hunt and Thomas's exact sense, produced at machine speed. The reuse gate (§5.3) and the prompt sections on reuse and replace-in-place (§5.1) target it directly.

Verification is asymmetric, and perception is unreliable. An agent's claim that its change works is cheap; observing the change working is not. Böckeler's push-the-autonomy study is blunt: agents "declared success in spite of red tests" *despite explicit instructions requiring*

passing tests, invented unrequested endpoints and business logic, silently filled requirement gaps with changed assumptions, and applied brute-force fixes to symptoms [11]. Her field notes on a background agent add base rates: across six runs of one real task, the agent duplicated existing functionality in four, found the reusable component in only two, and — because environment setup blocked the test suite — shipped a pull request with regressions it never saw [32]. The unreliability extends to human perception of the work: in a randomized controlled trial with 16 experienced open-source developers on 246 real tasks, allowing AI tools *increased* completion time by 19%, while the same developers believed afterwards they had been 20% faster [33]. At organisational scale, DORA's 2024 and 2025 reports find AI adoption associated with degraded delivery stability even as throughput turned positive, and frame AI as an amplifier: acceleration exposes whatever control systems a team lacks [34, 35]. Kim and Yegge codify the response as a trust-but-verify loop for production AI-assisted development [36], and Anthropic's engineering guidance stresses grounded feedback loops over open-loop generation [10, 37]. MAGENTRA's finishing checks (§5.5) are a mechanical demand for observation, with an explicit honest-failure path when observation is impossible; its live deliberation meter (§5.6) exists because perceived effort and measured effort demonstrably diverge.

Deliberation before action. LeCun's position paper on autonomous machine intelligence argues that capable agents should predict the consequences of candidate actions and choose among them against a cost, rather than react step by step [38]; the joint-embedding predictive architectures develop that program [39, 40]. MAGENTRA implements no world model, but it shares the instinct at the harness level: consequences are made visible before the edit (the import graph, §5.2), actions are priced by reversibility (§5.4), and the objective is clarified before work starts (§5.4).

3 Related Work

3.1 Agentic coding systems

SWE-agent introduced the agent-computer interface framing: the observation that an agent's performance depends heavily on the shape of the interface the harness gives it [41]. OpenHands (formerly OpenDevin) is an open platform for software-engineering agents with sandboxed execution and an event-stream architecture [42]. Aider, an open-source terminal agent, builds a repository map — a graph-ranked, token-budgeted summary of important files and symbols — precisely so the model reuses existing abstractions instead of reinventing them [43]; its leaderboards also document that the *output* interface matters — edit formats measurably shift model scores — extending the ACI thesis to the edit level. In the academic line, RepoGraph demonstrated that a repository-level code graph, plugged into four different agent systems, improves all of them on SWE-bench [44], and CodeAct showed that the shape of the action space itself (executable code versus structured tool calls) moves success rates by up to 20% [45]. MAGENTRA's knowledge subsystem is related work in this line: a multi-language index with five typed query operations — including impact analysis, not only retrieval — consumed by the agent, by write-time gates, and by the clarification step (§5.2). MAGENTRA keeps typed tool calls rather than CodeAct-style free code actions, both for simplicity and because judging an action's consequences before it runs (§5.4) requires knowing what the action *is*. Claude Code brought the terminal-agent form to wide use, with skills, hooks, and permission modes [46]; Cursor integrates agentic editing into an IDE [47]; Devin markets a fully autonomous software engineer, evaluated in a public

technical report [48]. MAGENTRA shares surface features with these tools — tools, permissions, on-demand procedures. What separates it is the deliberation loop around every turn — clarify before starting, demand evidence before ending — and the fact that all of it is open: where closed tools may run similar checks internally, here every check, prompt, and rule is visible and easily changeable by the user.

Benchmarks for this class of system include SWE-bench for repository-level issue resolution [49] and Terminal-Bench for end-to-end terminal tasks [50]. Both measure whether tasks complete; a younger family of benchmarks now measures what the completed task leaves behind (maintainability of and on agent code) and is discussed in §6. On the security side of harness design, Willison's analyses of prompt injection and the "lethal trifecta" (private data + untrusted content + external communication) argue that guardrails must be architectural rather than textual, since models cannot reliably rank instructions by their source [51, 52]; subsequent work formalises design patterns whose common principle is constraining an agent's authority once untrusted input has been ingested [53]. MAGENTRA's workspace-escape and protected-path rules (§5.4) are containment-shaped in exactly this sense — action-level constraints, not text filtering.

3.2 Reflection, verification, and memory

A separate line of work improves agent reliability through self-feedback atop the reasoning-and-acting substrate [54, 55]: Reflexion stores verbal reflections on task feedback in an episodic memory buffer [56]; Self-Refine iterates on a model's own critique [57]. The provenance of the feedback turns out to be decisive. Intrinsic self-correction — a model critiquing its own reasoning with no external signal — does not reliably work, and sometimes degrades performance [58], while self-debugging grounded in execution results does [59]. MAGENTRA's finishing checks came from the simpler observation that agents claim success they have not earned; the response was to trigger the critique mechanically, from observed facts about the turn (files changed, commands run, test doubles written), and to demand execution-grounded evidence — which is also what the self-correction literature indicates is required. The prompt catalog (§5.1) shares the prompts-are-structured-artifacts view of DSPy, which compiles declarative module signatures into optimized pipelines [60], though toward a different end: operator transparency and editability rather than automatic optimization.

Agent memory systems — MemGPT's OS-style paged context with self-editing memory [61], generative agents' memory stream with reflection and planning, whose ablations show each component necessary [62] — and skill-accumulation systems — Voyager's ever-growing library of verified executable skills [63], ExpeL's distilled cross-task insights [64], agent workflow memory's induced, reusable workflows [65], and CLIN's persistent memory of causal abstractions, which transfers zero-shot to new environments and tasks [66] — are the closest antecedents of the roadmap in §7: role agents that promote experience notes into accepted, portable skills. Multi-agent frameworks that assign software-engineering roles — MetaGPT [67], ChatDev [68] — share the role vocabulary but not the persistence: their roles are per-run personas, while the roadmap's role agents are long-lived artifacts whose training outlives any repository.

4 System Overview

MAGENTRA separates the interface from the agent (Fig. 1). The *engine* is a Node.js process owning everything agentic: sessions, tools, prompts, the knowledge index, permissions, and scheduling. A frontend drives it over newline-delimited JSON frames on stdio; the protocol

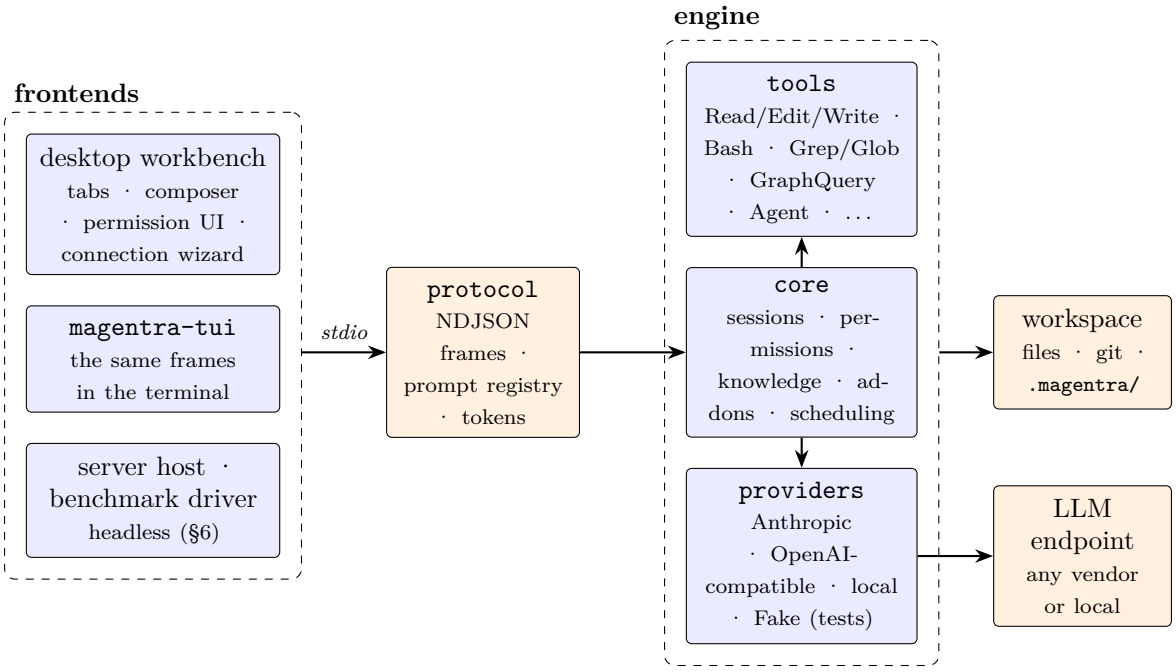

Figure 1: Process and package layout. Frontends drive the engine over newline-delimited JSON; the engine is provider-agnostic and treats the model endpoint as swappable. The protocol package also hosts the prompt registry (§5.1), so every prompt string crossing the boundary is a named, inspectable entity.

package defines the frame types and is the only shared dependency. Four frontends exist today: a desktop workbench (an Electron app with tabs, panes, a composer, permission dialogs, and connection management), a terminal interface (`magentra-tui`), a headless server host, and the benchmark driver of §6. Nothing in the engine knows which of them is driving it, so the same harness runs wherever a frontend can.

Three properties of this layout matter for the rest of the paper.

The model is swappable; the discipline is not. Providers are adapters behind one streaming interface; the engine defaults to OpenAI-compatible endpoints and runs unchanged against Anthropic models or a local server. Every mechanism in §5 lives on the engine side of that interface, so switching models does not switch off the discipline. A deterministic `FakeProvider` implements the same interface, which is how the harness logic is tested without any network dependency. Provider-agnosticism is maintained in small, concrete ways: a stream splitter reroutes the inline `<think>` reasoning that some locally served models leak into their answer text back into the thinking channel, and the retry layer recognises a long list of transient network errors (a sleeping laptop, a reconnecting VPN, a local server reloading a model) and narrates each retry to the user instead of showing a frozen spinner.

Prompts are data, not string literals. Every piece of prose the engine sends a model — system sections, reminders, background-call prompts — is registered through a `definePrompt` call with a stable identifier, a description of where it fires and what it costs, and its default text. Users can inspect the catalog, override any entry, or blank one to switch the corresponding behaviour off. This turns the harness's "built-in prompts" from folklore into configuration (§5.1).

State is plain files. Sessions, transcripts, settings, addons, and the knowledge index live under `.magentra/` in the workspace and the user's home directory, in human-readable formats. Sessions are resumable from their transcripts; on resume, any tool call that was interrupted mid-batch receives a synthetic error result first, so a replayed history is always one the provider will accept. Old transcripts and task outputs are pruned to user-set retention limits, so the harness bounds its own state as well. The engine's state directory is a protected path: outside the fully-autonomous mode, the agent cannot edit it without explicit confirmation (§5.4).

Sub-agents and scheduling. The engine spawns sub-agents through an Agent tool with a small typed registry: a general-purpose type with every tool, and read-only *explore* and *plan* types restricted to search and task-inspection tools. Every sub-agent's system prompt carries a result contract stating that its final message is raw data returned to the caller, not prose for a human, and all sub-agents share the root session's accounting ledger (§5.6). A scheduling substrate rounds out the runtime: a workflow runner executes deterministic orchestration scripts; an in-process scheduler handles recurring and one-shot wake-ups; a background manager tracks long-running jobs and notifies the agent when they finish; and for risky work, the agent can enter a git worktree under `.magentra/worktrees/` and work on an isolated copy of the repository. Users can also attach their own shell commands to lifecycle events (a hook that exits with code 2 blocks the action and returns its reason to the model), and external tool servers connect over the Model Context Protocol, joining the tool list like native tools.

5 The Harness as an Entropy-Control Loop

This section walks through one agent turn and the mechanisms that intervene in it. Figure 2 shows the lifecycle: the preparatory steps that run before the model acts, the per-action gates while it acts, and the finishing checks that decide whether the turn may end. The unifying rule was stated in §1: advisory where damage is recoverable, blocking only where it is not.

5.1 A prompt architecture with named parts

The system prompt is assembled from nine behaviour sections — identity, harness mechanics, communication, acting with care, git discipline, code style, task tracking, working method, and autonomy — each registered in the prompt catalog with a stable id (e.g. `system.code-style`), a channel (system, conditional section, in-turn reminder, or background call), and prose describing where it fires and what turning it off would change (Fig. 3). A section switched off resolves to the empty string and is dropped from assembly. The same registry holds every reminder, every background-call prompt, and every tool description in the engine, so the complete verbal surface of the harness is enumerable: there is no text the model reads that a user cannot also read, edit, or blank. The mechanics are plain. An override is a text file named after the prompt's id, kept in the user's home directory (prompts are tuned per user, not per repository) and picked up within a fraction of a second of being saved, with no restart. A blank file means *disabled*, which is distinct from deleting the file (reset to the shipped default), and disabling is honoured structurally: a background feature whose prompt is blank does not run at all, rather than running with empty instructions and still charging the user its latency.

Two sections carry most of the entropy discipline, and their content is a direct translation of the canon of §2 into imperatives a model acts on. The code-style section instructs, among other things:

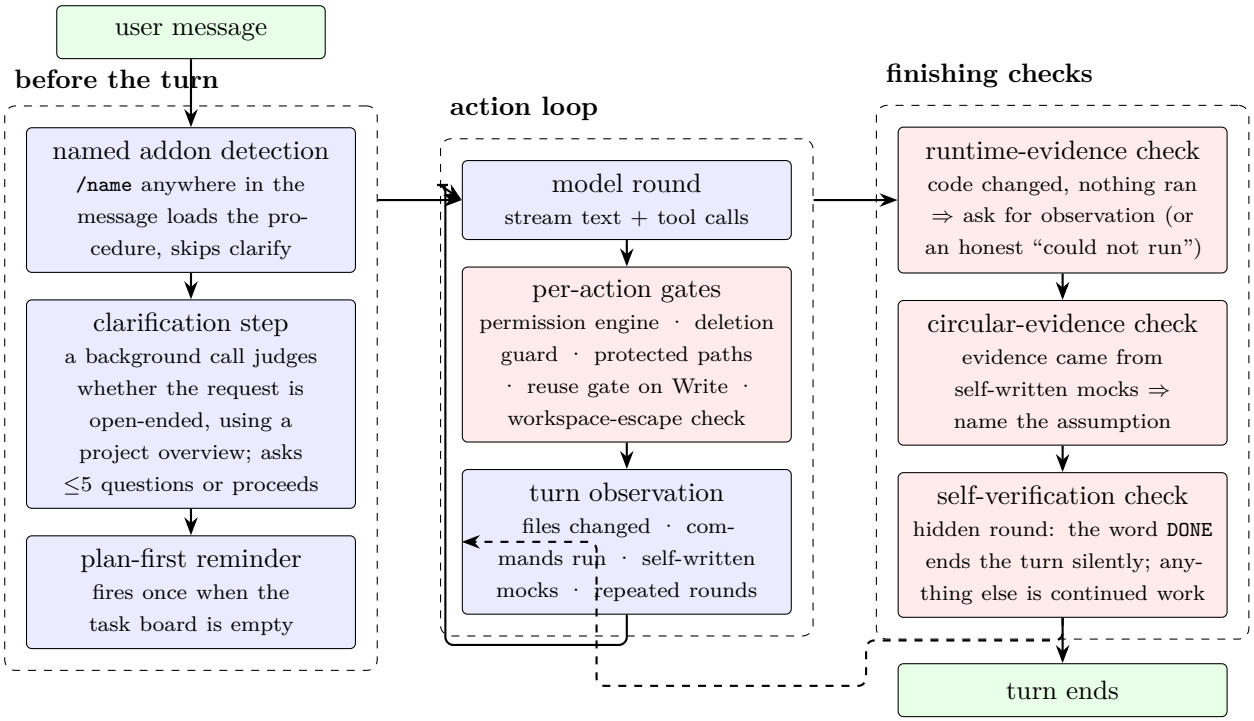

Figure 2: One agent turn. Preparatory steps run before the first action; per-action gates intervene during the loop; finishing checks stand between "the model stopped talking" and "the work is delivered." Dashed edge: a self-verification answer other than `DONE` is treated as continued work and re-enters the loop.

```
Reuse before you write. Before adding any function, type, helper or endpoint,
find out whether one already does that job -- search by what it DOES and by
the data it operates on, not only by the name you would have given it. [...]

Replace in place. When new code supersedes old code, the old code leaves in
the same change: migrate every call site, then delete the old definition and
anything that only existed to serve it. Never two versions side by side,
never a second name for the same job (fooV2, handleXNew, utils beside
helpers), never commented-out code left as a fallback.

Fix the mechanism, not the instance. Change the path every case goes
through, instead of bolting a condition onto the one spot you were looking
at. [...] Then fix all of it. The same cause in other call sites, branches
or copies belongs to this change, not a follow-up.
```

"Reuse before you write" is DRY [4] restated as a pre-condition; "replace in place" is the refactoring discipline of Fowler [17] compressed into a rule about what a single change must contain; "fix the mechanism" targets the special-case patches that Ousterhout identifies as complexity accumulation [18]. Prompt text alone, however, is a request, not a mechanism — models comply unevenly, and compliance degrades with context pressure. The rest of §5 describes the machinery that backs these requests with checks.

Workspaces can add a binding standards file: when `STANDARDS.md` exists, its content is injected under a header declaring it rules rather than suggestions, overriding the default style

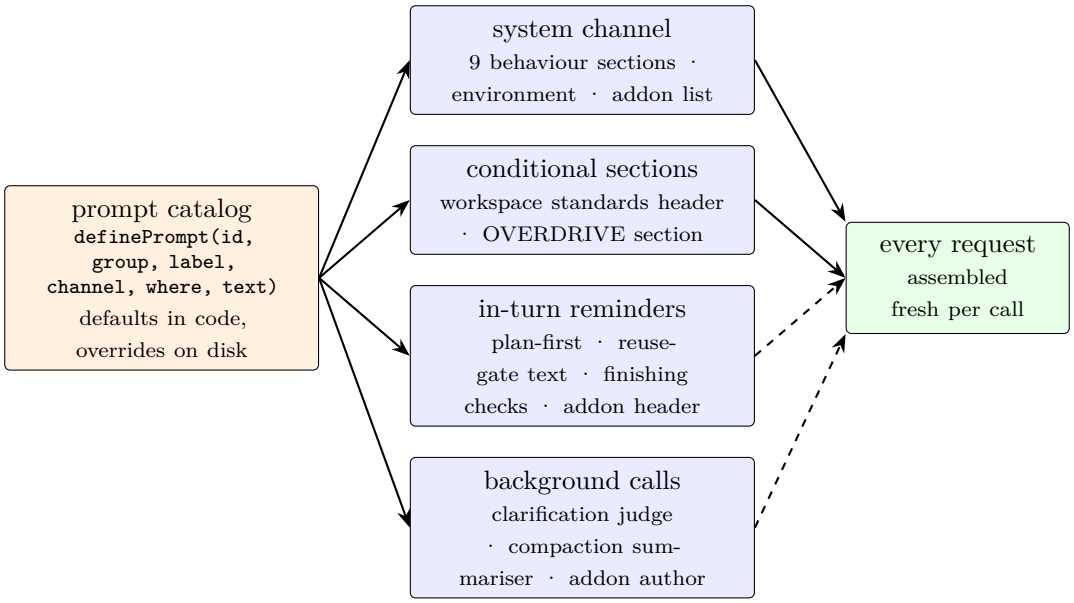

Figure 3: Prompt assembly. Every string the engine sends a model is a registered entry with an id, a channel, and an account of its cost; users override or blank entries per workspace. Dashed edges fire conditionally.

guidance where they conflict.

5.2 Structural self-knowledge: the import graph

The engine builds an import graph of the workspace: files are nodes, and resolved import or include relations are edges. Per-language extractors cover JavaScript/TypeScript, Python, C/C++, Ruby, PHP, JVM languages, Go, and Rust. The graph persists beside the workspace state and is rebuilt when stale. On top of it sit standard graph analyses. Relevance ranking is personalised PageRank in which the random walk follows import edges in both directions — weight 1.0 toward what a seed file uses and 0.5 toward what uses it — so relevance flows both to dependencies and to callers; `slice` then admits the seeds and fills the remaining token budget greedily by descending rank. Impact analysis is reverse reachability: every module that transitively imports a target, grouped by hop distance — which makes the "change amplification" Ousterhout describes as a symptom of complexity [18] a number the agent can ask for before editing. Structural summaries use the Hopcroft–Tarjan computation of articulation points and bridges, so "which files hold this repository together" has its ordinary graph-theoretic meaning.

The agent reaches this through one tool, `GraphQuery`, with five operations (Fig. 4):

`slice` ranked minimal context for a topic: given seed files and/or keywords, personalised PageRank selects the most relevant files that fit a token budget, returned with the edges among them — a structural alternative to guessing which files to open;

`blast` the *blast radius* of a prospective change: the transitive set of modules that import the given files, grouped by hop distance — what breaks if these files change;

`deps` a file's forward dependency closure, external packages listed separately;

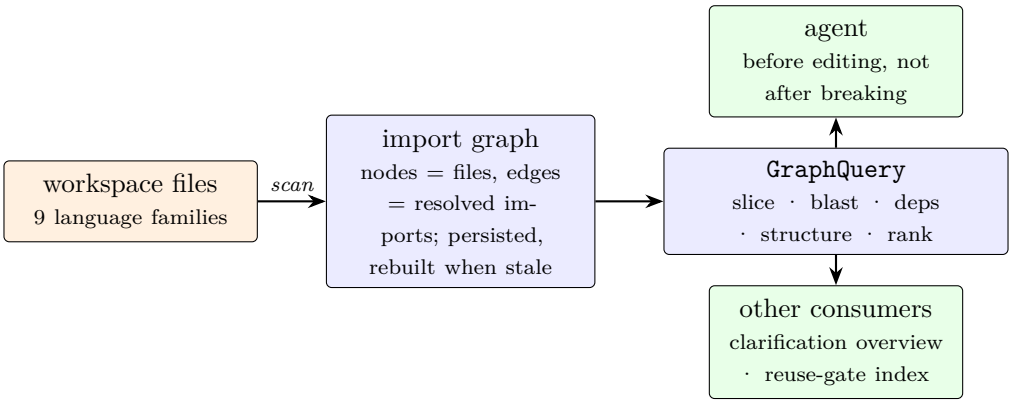

Figure 4: The knowledge subsystem. One index serves the agent's impact queries, the clarification step's project overview, and the reuse gate's symbol search.

`structure` a repository overview: top files by centrality, articulation points, bridges, component counts;

`rank` global or seed-personalised centrality.

The prompt sections direct the agent to prefer these queries over exploratory file reading and to run `blast` before editing anything widely imported. The intent is economic and cognitive at once: a structural answer that costs a few hundred tokens replaces speculative reads costing tens of thousands — exactly the focused-input condition the context-rot results reward [24] — and, more importantly for entropy, it makes the *consequences* of an edit visible before the edit exists. The gap it addresses is independently documented: SmellBench's evaluation of code agents on repository-level refactoring attributes their failures to "a focus on local code smells and a lack of cross-file understanding" [69], and CodeTaste finds agents execute detailed refactoring instructions well but fail to *discover* the refactorings humans actually chose [70] — discovery is precisely what a queryable structural index scaffolds. This is the harness-level analogue of action-consequence modelling discussed in §2: not a learned world model, but a queryable, always-current model of one world the agent acts in, namely the dependency structure of the repository.

Sub-agents inherit the same graph, and its summary (file counts plus most central files) is what the clarification step reads to ground its questions in the actual project rather than generic ones.

5.3 The reuse gate: search before write

The most direct anti-duplication mechanism intercepts `Write` calls that would create a new source file. The engine maintains a symbol index over the workspace and a per-session *search log*: search-shaped tools (Grep, Glob, GraphQuery) declare their search terms through a typed tool-interface capability, and every call's terms are recorded. When a new-file Write arrives, a pure decision procedure (Fig. 5) checks, with no model call and typically in milliseconds, whether the file's symbols and name resemble code that already exists, and whether the session shows any evidence the agent looked for it: a related search, or a read of one of the closest matching files. Similarity is simple and inspectable by design: identifiers are split into tokens at case and punctuation boundaries, generic words (`get`, `util`, `helper`) are dropped, and two names are

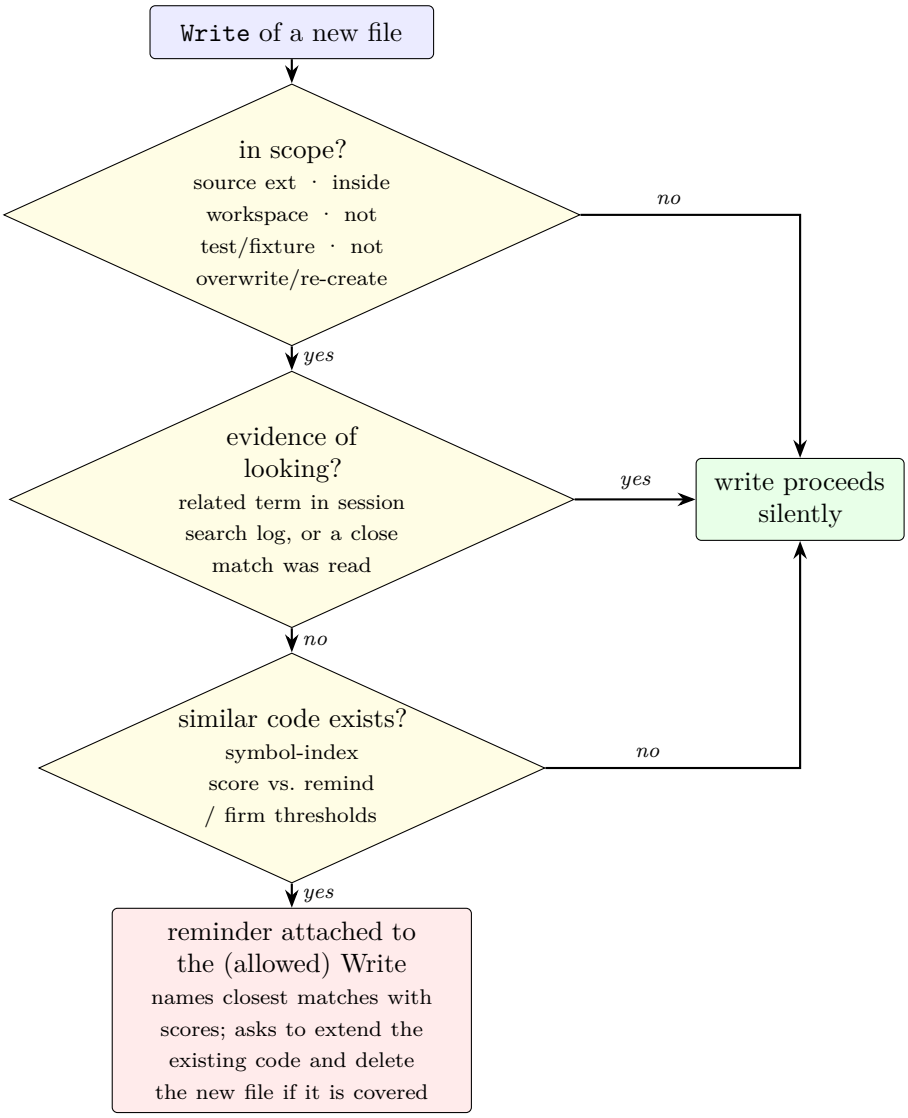

Figure 5: The reuse gate, evaluated without any model call. Test files, overwrites, re-creations of files read this session, and out-of-workspace paths are exempt. Every uncertainty fails open: the gate must never wrongly block a write.

compared by token-set overlap with a small bonus when their first or last tokens agree; an exact normalised name scores 1.0.

Two design decisions matter. First, the gate *reminds rather than blocks*: the Write is allowed, and the reminder — naming the closest matches with similarity scores, and asking the agent to read them and fold the new code into an existing home if one covers it — is attached to the successful result, so the signal reaches the model without ever stalling the work. Second, the gate *fails open on every uncertainty*: an unreadable index, an unparseable file, any internal error passes the write through. A duplicate file is recoverable; a harness that unpredictably blocks legitimate work teaches both the model and the user to distrust it, which is worse.

One property of the design deserves emphasis: an agent that follows the prompt's "search by what it does" instruction never sees the gate at all, because its searches populate the log and

satisfy the evidence check. The gate exists for the turn where context pressure or haste erodes the discipline. It is the mechanism standing behind the request.

The stakes of that turn are quantified in the literature. Duplication is not cosmetic: half of code clones in industrial systems end up changed inconsistently, and a measurable fraction of those inconsistencies are faults [30]. And the no-search-then-write failure is common in practice: in Böckeler's six runs of a background agent on one task, the agent re-implemented existing functionality in four runs and found the reusable component in only two [32] — close to the exact scenario the evidence check is built to interrupt.

5.4 Deliberation before and during action

Clarification before starting. The clarification step exists first of all for the user: an agent that silently guesses the shape of an open-ended request ("build a game," "improve this app") and delivers the wrong thing wastes the user's time and trust, while one that asks two or three sharp questions makes the user a participant in the plan before any work is spent. Before acting on such a request, a background model call (the same model, with a separate small prompt) judges whether guessing the unstated choices wrong would force a redo. Only then does the harness put up to five concrete, multiple-choice, decision-changing questions to the user, each grounded in a short overview of the project (the graph summary of §5.2, or a look at the README and manifests when the graph has nothing). Grounding matters: generic questions read as ceremony, while questions that name the project's actual stack and options read as competence. The prompt is explicit that the codebase shows what *exists*, not what the user now *wants*: knowing the project is a reason to ask a sharper question, not to skip asking. The answers ride into the turn as requirements. The step is strictly fail-open — any inference error proceeds without questions — and is skipped when the message names an addon, whose procedure already answers "what shape should this work take." The cost of the extra round is supported by the clarification literature: ClarifyGPT, which detects ambiguity by sampling divergent solutions and then asks targeted questions, lifts GPT-4 from 70.96% to 80.80% Pass@1 on MBPP-sanitized [71] — roughly ten points recovered by asking before guessing. Magentra's variant judges ambiguity against an overview of the actual workspace rather than sampling divergence.

Permissions by reversibility. A call is judged by what it can do, not by which tool makes it, and every request resolves in a fixed order: deny rules, then the protected-path guard, then the deletion guard, then allow rules, then the default, which is to allow. Reads and workspace-local edits therefore run without asking; the prompts spend their weight on making that freedom careful rather than slow. Two kinds of target always confirm. The first is deletion of a file, folder, or worktree — including deletions issued through shell commands, which the Bash tool recognises by shape: `rm`, `git clean`, `git push -force`, `find -delete`, destructive `mv`, and so on, distinguishing even the force-delete `git branch -D` from the safe `-d`. The second is an edit to a protected path (`.magentra/`, the engine's own state, or a `.env` secrets file). File edits that escape the workspace root, such as a shell profile or an SSH configuration, also ask first; that is the shape a prompt-injection attempt takes [51]. "Always allow" grants are scoped to a command's *shape* (`git push`, not this exact push, and never all of `git`), and the dialog shows the scope, so a grant is never broader than what the user saw. One asymmetry is deliberate: convenience grants are by shape, but only a *literal* grant can override the deletion guard — a shape grant earned by a benign `git push` never lets a later `git push -force` skip its confirmation.

Freshness before edits. An Edit, or a Write over an existing file, fails unless the file was read this session and is unchanged on disk since (checked by modification time and size). This protects against two different mistakes: editing from a stale memory of the file, and silently overwriting changes a human made in parallel. Every applied change is emitted as a unified diff — what the user reviews, and what feeds the session's lines-changed ledger.

Fully autonomous mode (OVERDRIVE). An optional session mode removes the remaining confirmations for unattended work; user-written deny rules still apply, the mode's prompt section transfers responsibility to the agent's own care, and before each turn the engine snapshots the working tree (a git stash commit) so in-workspace deletions stay recoverable.

Stall detection and steering. The action loop fingerprints each round (tool calls plus results); consecutive identical rounds trigger a reminder to change strategy, and after two spent pivots the harness stops and asks the user rather than burning tokens on a loop. The user can steer mid-run: queued guidance lands at the next message boundary, re-arms the self-verification check, and refunds spent pivots — the user changed the game, so the old stall evidence is void.

5.5 Finishing checks: the demand for evidence

The turn does not end because the model stopped talking. Three checks stand at the bottom of the loop (Fig. 2), each triggered by observed facts about the turn, not by model self-report. The session records which files were changed by successful `Write`/`Edit` calls, whether any command was executed, and whether the text written this turn contains markers of self-authored test doubles (mocking libraries by name, `class Fake...`, `def mock_...`). Only successful calls count: a refused Write or a command that never ran changed nothing, and crediting it would let a turn pass the evidence check with calls that did nothing. The first two checks are one prompt with two conditional shapes on purpose — they were once separate prompts firing on opposite halves of "did anything run," and disabling either one silently uncovered its half.

The design rests on a finding from the self-correction literature: intrinsic self-critique — asking a model whether its own work is correct, with no external signal — does not reliably help and can hurt [58], while critique grounded in execution results does [59]. The checks therefore never ask the model to re-judge its reasoning; they confront it with facts the harness observed and demand evidence of a kind the model cannot generate by introspection. Böckeler's finding that agents declared success over red tests *despite instructions demanding passing tests* [11] is the cautionary tale for instruction-only designs; the checks are what instructions backed by mechanism look like.

Runtime evidence. If source files changed and no command ran, nothing about the change has been *observed* — so a reminder fires, once per turn, naming the changed files and asking the agent to work down a list: run the project's fast gate (while noting that passing it "proves the code parses, not that it behaves"); execute the changed path and the callers it reaches; if needed, write a throwaway driver in the system temp directory and delete it the same turn; judge against something readable (exit codes, stdout, a log line); and report what was run and observed. The closing paragraph is, by design, load-bearing: if the change genuinely cannot be executed on this machine, *stopping and saying so* — naming the closest thing that did run and what stays unverified — is a fully correct ending. A check that demanded green results would manufacture the exact failure it exists to catch: a fabricated verification.

Circular evidence. The second shape of the same question covers the turn that *did* run something, where what ran was a stand-in the agent wrote itself. A mock is a model of the dependency authored by the same understanding that authored the code; the two agree by construction, so a passing check proves self-consistency, not correctness. The warning asks the agent to say where each replaced contract came from — and "I assumed it" is named as the thing to fix. The marker list is tuned for precision over coverage: a missed double costs one silent turn, while a false positive spends a round trip and teaches the model to skim the reminder, which is worse than never sending it.

Self-verification. In autonomous mode, a turn that made tool calls ends with one final round: the model is asked whether every part of the original query is fully handled and nothing unnecessary was left behind (scratch files, duplicated helpers, abandoned attempts). The word `DONE` ends the turn; anything else is treated as continued work and streamed to the user. Mid-run user steering re-arms the check, so "done" is always judged against the latest statement of what the user wants.

Two smaller reminders complete the family: a missing-wrap-up reminder when the agent stops working without summarising what changed and what was verified, and continuation reminders that resume output mid-character when a response is cut off at a token limit rather than restarting it.

5.6 Deliberation made visible

The harness treats "how much thinking is happening" as a user-facing quantity. Session accounting distinguishes three numbers that are easy to conflate. Let invocations $i = 1, \ldots, n$ be every model call made since the current turn opened — the session's own, its background calls, and its sub-agents' — with input and output token counts in_i and out_i. Then

$$B(t) = in_{\text{latest}}, \qquad D(t) = \sum_{i=1}^{n} out_i, \qquad T_{\text{turn}} = \sum_{i=1}^{n} (in_i + out_i),$$

where $B(t)$ is the current context occupancy (the input of the latest conversational request — a point-in-time measure, not a running total), $D(t)$ is the live *deliberation* figure counted while the agent works, and T_{turn} is the turn's total billed usage, reported when the turn ends. Cumulative usage per model accumulates separately across the session. The workbench shows $D(t)$ ticking during a turn and reports the context breakdown (system prompt, tools, addons, messages) on demand. The practical effect is that the cost of the mechanisms in this section — the extra prompt text, the occasional extra round — is not hidden from the person paying for it; an operator who finds a check too expensive can see the price and blank the prompt that causes it. Measurement matters here because perception demonstrably does not: developers in the METR trial believed AI had made them 20% faster while it had made them 19% slower [33], and DORA finds 80%+ perceived productivity gains alongside degraded delivery stability [35].

Context management belongs to the same ledger. When the window approaches a user-set limit, a background model call summarises the older history into a fixed, decision-preserving structure: task state and goal, decisions and why, files touched with paths, open items. Background calls like this run on a cheaper configured model by default (the clarification judgment is the deliberate exception and uses the main model), and their private prompts never enter the conversation window. The summary replaces the compacted span, wrapped in text that

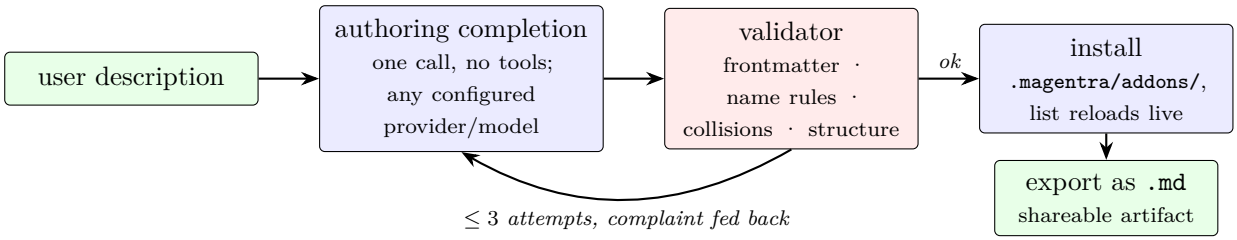

Figure 6: Addon generation: a bounded generate–validate–repair loop, not an agent. The validator's rejection text becomes the next attempt's feedback. Installed addons are immediately invokable; exported addons are plain markdown files.

explicitly prevents the agent from reading compaction as a signal to wrap up. This is the structured, focused-input form the long-context results of §2 favour [23, 24], and the same recall-then-precision doctrine Anthropic's context-engineering guidance recommends [26].

5.7 Addons: procedures on demand

An addon is a markdown procedure with a name and a one-line description. Only the list of names and descriptions is included in the system prompt; a body is paid for only when invoked, either by the agent reaching for it or by the user naming it with a slash anywhere in a message ("`use /magentron on this`"). Addons load from three tiers — built-ins shipped in code, the user's home directory, the workspace — with name collisions resolved in that order, so a workspace can override a built-in outright. A directory addon can carry sibling reference files and runnable scripts, which the loader advertises so the agent knows what it may read or execute.

The invocation header fixes the precedence explicitly: an addon's instructions outrank the agent's defaults, and the *user* outranks the addon. One shipped built-in, `magentron`, packages the entropy discipline of §5.1 as an explicit three-phase procedure (map what exists before writing; replace in place; audit coherence afterwards) for changes where a broken edit costs more than the reconnaissance.

This loading economics has since become an ecosystem pattern: Anthropic's Agent Skills — folders of instructions, scripts, and resources whose names are pre-loaded while bodies load on match, under the name "progressive disclosure" — shipped in October 2025 and were later opened as a cross-platform standard with a public skills directory [72]. We note the convergence as parallel design in the same period rather than influence in either direction; its significance for §7 is that ecosystems for *sharing procedures* demonstrably form. At the workspace-instruction level the ecosystem has likewise standardised: the `AGENTS.md` convention — a root-level instruction file for coding agents, formalised in 2025 and since adopted by tens of thousands of repositories and stewarded by the Linux Foundation's Agentic AI Foundation [73] — plays the role Magentra's binding standards file plays in §5.1.

Addons close a loop the roadmap of §7 extends: the engine can *author* them. Given a description, a single non-agentic completion drafts the addon file; a validator checks its frontmatter, name collisions, and structure; on rejection the validator's complaint is fed back for up to three attempts. Accepted addons install into the workspace tier and are invokable on the next request; any addon can be exported as a plain markdown file and shared. This is deliberately a compiler loop, not an agent: a generate–validate–repair cycle with a hard attempt bound (Fig. 6).

6 Planned Evaluation

Benchmark evaluation is future work. The plan has two axes, because the harness's claims have two parts — *does the task complete*, and *what is left behind*.

Completion. The repository contains a working Terminal-Bench adapter that runs the *unmodified* engine as an installed agent: the same engine bundle the packaged app ships is uploaded into each task container and driven over its normal NDJSON protocol by a thin driver — a frontend, not a fork — under Harbor, the containerized evaluation framework released with Terminal-Bench 2.0 [50]. The adapter configures only the model connection and the autonomous mode (a benchmark has no user to ask or to clarify with); prompts, tools, knowledge graph, reuse gate, permission engine, and turn caps are identical to the shipped product. Initial runs validate the setup; full evaluation and a leaderboard submission can follow in future work.

What is left behind. A younger family of benchmarks measures maintainability directly, and future evaluation can draw on it: SWE-CI operationalizes maintainability as functional correctness on *future* modifications across real development histories [74]; the CodeThread methodology measures how much harder agent-written code is for the *next* agent [31]; SmellBench scores repository-level smell elimination [69]; and CodeTaste tests whether refactoring *choices* match human ones [70]. Alongside these external instruments, the harness's own observability yields per-session metrics at no extra cost: reuse-gate fire rates and their outcomes (was the new file kept, or folded into an existing home), lines added versus removed, and how often finishing checks convert a would-be silent ending into a verification run or an honest statement of what remains unverified. The session ledger of §5.6 already collects the raw quantities.

7 Roadmap: Trainable, Shareable Role Agents

The mechanisms of §5 are stateless across projects: the reuse gate re-derives its index per workspace, and the prompt catalog is configuration, not experience. The next stage of Magentra — implemented in part and under test at the time of writing, with results deliberately not claimed here — extends the addon substrate into *role agents* that accumulate experience over time and carry it between repositories.

Role agents. A role agent is a named, persistent configuration of the harness — a developer, a designer, a reviewer — owning its own skill set and its own experience record. Where today's addons are procedures a user writes or generates once, a role agent's skills are meant to *grow* out of its work.

Experience notes and promotion. During real sessions, the agent records notes: corrections received, approaches that failed, feedback from its user. Notes are provisional by construction; lessons that prove themselves are promoted, in stages, into skills the agent keeps (Fig. 7). The exact staging and promotion criteria are what the test phase is deciding, so we describe the shape rather than the parameters; the guiding rule carries over from §5.3 — a bad lesson must be cheap to shed.

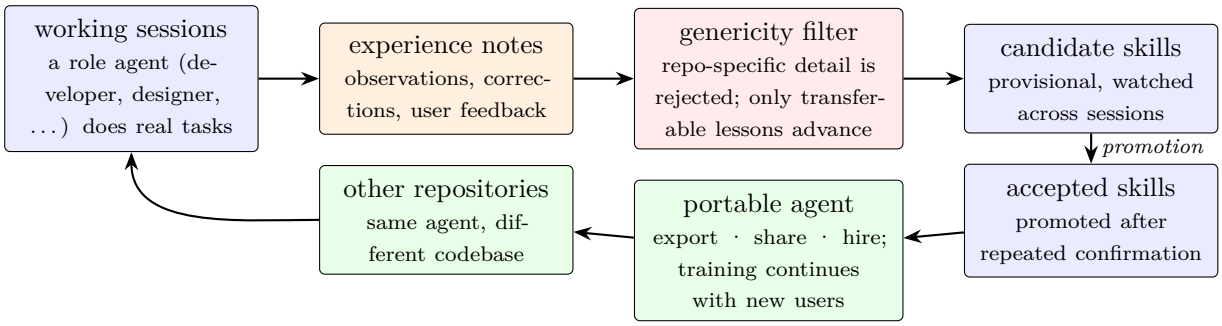

Figure 7: The intended experience pipeline for role agents. Notes taken during real work pass a genericity filter; surviving lessons become candidate skills and are promoted to accepted status only after repeated confirmation. The resulting agent is a portable artifact whose training continues wherever it is used. Under test; no results are claimed.

The genericity filter. The load-bearing constraint is that role agents learn *generic* lessons, not repository facts. "Be careful to choose data types that will not break the application when declaring variables" is admissible; "this project keeps validators in `utils/`" is not — it is true of one repository and would be entropy inside the agent when carried to the next. Repository-specific knowledge already has homes (the workspace's standards file, its addons, its generated map); the role agent is precisely the container for what transfers. This separation is what makes the same developer agent usable across many repositories — and it is also a privacy boundary: an exported agent carries lessons, not code. The design bet has direct support in the continual-learning literature: CLIN, whose persistent memory stores causal *abstractions* rather than environment facts, transfers zero-shot to new environments and tasks and outperforms reflection-only agents by wide margins [66]; ExpeL and agent workflow memory likewise show that distilled, reusable units of experience improve long-horizon performance without weight updates [64, 65]. What the roadmap adds to that lineage is gatekeeping: unvetted accumulation is treated as an entropy source in its own right.

Identities and privilege levels. A role agent is also an *identity*. Businesses and security practice run every human and service account under a defined privilege level — least privilege, role-based access control — and agents that act on real systems need the same treatment: a named identity, a bounded set of rights, and an audit trail of what was done under them. Magentra will support this idea directly: role agents are the natural unit to carry an identity, and the permission engine of §5.4, which already prices every action and records every grant, is the natural enforcement point for per-agent privilege levels. The same identity substrate is what the agent-economies literature says a trustworthy exchange requires [75].

Role grounding. The developer role's learnable signal is the one this whole paper is about — corrections, verification outcomes, reuse decisions. The designer role has its own emerging literature: ratings-based preference learning discards the rationale designers attach to critique, and models trained on workflow-native feedback (comments, sketches, direct manipulation) outperform ranking-trained baselines [76]. A persistent designer agent is the continual-learning extension of that finding: rationale-bearing feedback, accumulated across projects, promoted into taste.

Sharing, hiring, and community training. Because an agent's skills are plain, inspectable artifacts on the addon substrate, a trained agent is exportable the way a single addon is today (§5.7). The intended economy follows: users who train a developer agent through months of work can publish it; others can hire or import it and continue its training in their own projects; a designer agent accumulates taste from the feedback loops it survives. Procedure-sharing ecosystems already exist — the open Agent Skills standard grew a public directory within months [72] — and what the roadmap adds is the training history: a shared role agent carries promoted experience and its provenance, not just static instructions. The design questions this raises are the subject of an emerging literature. Tomasev et al. frame agent economies along two axes — intentionally designed versus emergent, and permeable versus isolated from the human economy — and argue for deliberate design with identity, reputation, and accountability infrastructure before a vast, highly permeable economy emerges on its own [75]; testbeds for agent labor markets are beginning to study how allocation and enforcement rules shape trade and quality [77]. How such an exchange should be designed — where it sits on those axes, how permeable it should be to the wider economy, and what a hired agent is permitted to do — is an open design question that belongs to the test phase; we do not settle it here. What holds in every variant is the need for the trust infrastructure Tomasev et al. name: provenance and reputation for traded agents are open problems, including adversarial ones — a shared agent is a prompt supply chain [52]. In Karpathy's terms, it is a step toward the missing "GitHub of Software 2.0" — infrastructure for sharing and versioning the new artifact class [21]. These questions, like the pipeline itself, are under active test.

8 Limitations

The central limitation is stated throughout: this paper describes and motivates mechanisms; it does not measure them. Fire rates, false-positive costs, and end-to-end effects on code quality are unquantified until the evaluation of §6 lands.

Beyond that: the import graph is structural, not semantic, so code linked only dynamically is invisible to it; the reuse gate's similarity is lexical; the security guards are containment heuristics, not proofs [52, 53]; and the roadmap of §7 is a design under test.

9 Conclusion

The software-engineering literature spent four decades documenting how codebases decay and what discipline resists the decay. Agentic coding changes both sides of that account: an agent can produce entropy at machine speed, and a harness can apply discipline more consistently than human attention sustains. Magentra builds that discipline into a working, open tool: structural knowledge, write-time gates, clarification before work, and end-of-turn evidence demands, each traceable to a named decay mode, with costs visible to the person paying them and every mechanism open to inspection and change. A harness of this kind reduces entropy; it does not eliminate it. Future work proceeds on two fronts: benchmark evaluation of the present system (§6), and the role-agent roadmap of §7 — with fuller answers to entropy in agentic coding a continuing research direction at Kalai Labs.

Acknowledgments

MAGENTRA came out of the author's PhD work at Hacettepe University, Ankara, Türkiye (Computer Engineering), within studies on agentic AI systems, agentic surveillance, agentic zero-shot prediction mechanisms, and agentic harnesses.

AI Credibility

Parts of the implementation were produced with AI assistance, under the author's direction and review; every such contribution was declared by the author during the work. All claims are the author's own, and full responsibility for the content rests with the author.

References

[1] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, Ge Li, et al. A survey on code generation with LLM-based agents. arXiv:2508.00083, https://arxiv.org/abs/2508.00083, 2025.

[2] Meir M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980. https://doi.org/10.1109/PROC.1980.11805.

[3] Andrew Hunt and David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, Boston, MA, 1999. https://books.google.com/books?vid=ISBN9780201616224.

[4] David Thomas and Andrew Hunt. *The Pragmatic Programmer: Your Journey to Mastery, 20th Anniversary Edition*. Addison-Wesley, Boston, MA, 2nd edition, 2019. https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/.

[5] James Q. Wilson and George L. Kelling. Broken windows: The police and neighborhood safety. *The Atlantic Monthly*, 249(3):29–38, 1982. https://www.theatlantic.com/magazine/archive/1982/03/broken-windows/304465/.

[6] David Lorge Parnas. Software aging. In *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, pages 279–287, Sorrento, Italy, 1994. IEEE Computer Society Press. https://dl.acm.org/doi/10.5555/257734.257788.

[7] Ward Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, pages 29–30, Vancouver, BC, Canada, 1992. ACM. https://doi.org/10.1145/157709.157715.

[8] William Harding and Matthew Kloster. Coding on Copilot: 2023 data suggests downward pressure on code quality. Technical report, GitClear, January 2024. https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality, 2024. Industry report, not peer-reviewed. Accessed 2026-08-02.

[9] William Harding. AI copilot code quality: 2025 data suggests 4x growth in code clones. Technical report, GitClear. https://www.gitclear.com/ai_assistant_code_quality_2025_research, 2025. Industry report, not peer-reviewed. Accessed 2026-08-02.

[10] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic engineering blog, December 2024. https://www.anthropic.com/engineering/building-effective-agents, 2024. Accessed 2026-08-02.

[11] Birgitta Böckeler. How far can we push AI autonomy in code generation? martinfowler.com, August 5, 2025. https://martinfowler.com/articles/pushing-ai-autonomy.html, 2025. Accessed 2026-08-02.

[12] Meir M. Lehman and Laszlo A. Belady. *Program Evolution: Processes of Software Change*. Academic Press, London, 1985. https://books.google.com/books?vid=ISBN9780124424401.

[13] Frederick P. Brooks, Jr. *The Mythical Man-Month: Essays on Software Engineering, Anniversary Edition*. Addison-Wesley, Boston, MA, 1995. https://books.google.com/books?vid=ISBN9780201835953.

[14] Frederick P. Brooks, Jr. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. https://doi.org/10.1109/MC.1987.1663532.

[15] Dewayne E. Perry and Alexander L. Wolf. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4):40–52, 1992. https://doi.org/10.1145/141874.141884.

[16] Ahmed E. Hassan. Predicting faults using the complexity of code changes. In *Proceedings of the 31st International Conference on Software Engineering (ICSE '09)*, pages 78–88. IEEE, 2009. https://doi.org/10.1109/ICSE.2009.5070510.

[17] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, Boston, MA, 2nd edition, 2018. https://martinfowler.com/books/refactoring.html.

[18] John K. Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, Palo Alto, CA, 2nd edition, 2021. https://books.google.com/books?vid=ISBN9781732102217.

[19] Steve McConnell. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press, Redmond, WA, 2nd edition, 2004. https://books.google.com/books?vid=ISBN9780735619678.

[20] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, Upper Saddle River, NJ, 2008. https://books.google.com/books?vid=ISBN9780132350884.

[21] Andrej Karpathy. Software 2.0. Blog post, Medium, November 11, 2017. https://karpathy.medium.com/software-2-0-a64152b37c35, 2017. Accessed 2026-08-02.

[22] Birgitta Böckeler. The role of developer skills in agentic coding. martinfowler.com, March 25, 2025. https://martinfowler.com/articles/exploring-gen-ai/13-role-of-developer-skills.html, 2025. Accessed 2026-08-02.

[23] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. https://doi.org/10.1162/tacl_a_00638.

[24] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, July 14, 2025. https://www.trychroma.com/research/context-rot, 2025. Accessed 2026-08-02.

[25] Andrej Karpathy. +1 for "context engineering" over "prompt engineering". Post on X, June 25, 2025. https://x.com/karpathy/status/1937902205765607626, 2025. Accessed 2026-08-02.

[26] Anthropic. Effective context engineering for AI agents. Anthropic engineering blog, September 29, 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025. Accessed 2026-08-02.

[27] Yue Liu, Ratnadira Widyasari, Yanjie Zhao, Ivana Clairine Irsan, Junkai Chen, and David Lo. Debt behind the AI boom: A large-scale empirical study of AI-generated code in the wild. arXiv:2603.28592, https://arxiv.org/abs/2603.28592, 2026. Accessed 2026-08-02.

[28] GitClear. The maintainability gap: 2026 AI code quality research. Technical report, GitClear and GitKraken. https://www.gitclear.com/the_ai_code_quality_maintainability_gap, 2026. Industry report, not peer-reviewed. Accessed 2026-08-02.

[29] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. Investigating the smells of LLM generated code. arXiv:2510.03029, https://arxiv.org/abs/2510.03029, 2025. Extended version of a paper published at ICCBDCS 2025.

[30] Elmar Juergens, Florian Deissenboeck, Benjamin Hummel, and Stefan Wagner. Do code clones matter? In *Proceedings of the 31st International Conference on Software Engineering (ICSE '09)*, pages 485–495. IEEE, 2009. https://doi.org/10.1109/ICSE.2009.5070547.

[31] Shaswat Patel, Betty Li Hou, Arun Purohit, Kai Xu, Jane Pan, He He, and Valerie Chen. Is agent code less maintainable than human code? arXiv:2606.21804, https://arxiv.org/abs/2606.21804, 2026.

[32] Birgitta Böckeler. Autonomous coding agents: A Codex example. martinfowler.com, June 4, 2025. https://martinfowler.com/articles/exploring-gen-ai/autonomous-agents-codex-example.html, 2025. Accessed 2026-08-02.

[33] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv:2507.09089, https://arxiv.org/abs/2507.09089, 2025. METR randomized controlled trial.

[34] DORA. Accelerate state of DevOps report 2024. Google Cloud. https://dora.dev/research/2024/dora-report/, 2024. Accessed 2026-08-02.

[35] DORA. 2025 DORA report: State of AI-assisted software development. Google Cloud, September 24, 2025. https://dora.dev/, 2025. Accessed 2026-08-02.

[36] Gene Kim and Steve Yegge. *Vibe Coding: Building Production-Grade Software With GenAI, Chat, Agents, and Beyond*. IT Revolution Press, 2025. https://itrevolution.com/product/vibe-coding-book/.

[37] Anthropic. Effective harnesses for long-running agents. Anthropic engineering blog. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025. Accessed 2026-08-02.

[38] Yann LeCun. A path towards autonomous machine intelligence, version 0.9.2. OpenReview, https://openreview.net/pdf?id=BZ5a1r-kVsf, 2022. Position paper. Accessed 2026-08-02.

[39] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023. arXiv:2301.08243, https://arxiv.org/abs/2301.08243.

[40] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024. arXiv:2404.08471, `https://arxiv.org/abs/2404.08471`. V-JEPA.

[41] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. arXiv:2405.15793, `https://arxiv.org/abs/2405.15793`.

[42] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.16741, `https://arxiv.org/abs/2407.16741`. Formerly OpenDevin.

[43] Paul Gauthier. Aider: AI pair programming in your terminal. `https://github.com/Aider-AI/aider`, 2024. Open-source software; repository map documented at `https://aider.chat/docs/repomap.html`. Accessed 2026-08-02.

[44] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.14684, `https://arxiv.org/abs/2410.14684`.

[45] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.01030, `https://arxiv.org/abs/2402.01030`.

[46] Anthropic. Claude code. `https://www.anthropic.com/claude-code`, 2025. Agentic coding tool. Accessed 2026-08-02.

[47] Anysphere, Inc. Cursor: The AI code editor. `https://cursor.com`, 2023. Commercial software, first released March 2023. Accessed 2026-08-02.

[48] Cognition AI. SWE-bench technical report. `https://cognition.com/blog/swe-bench-technical-report`, 2024. Technical report on Devin, March 15, 2024. Accessed 2026-08-02.

[49] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770, `https://arxiv.org/abs/2310.06770`.

[50] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv:2601.11868,

https://arxiv.org/abs/2601.11868, 2026. Benchmark first released May 2025 by the Laude Institute and Stanford; leaderboard at https://www.tbench.ai.

[51] Simon Willison. Prompt injection attacks against GPT-3. Blog post, September 12, 2022. https://simonwillison.net/2022/Sep/12/prompt-injection/, 2022. Accessed 2026-08-02.

[52] Simon Willison. The lethal trifecta for AI agents: Private data, untrusted content, and external communication. Blog post, June 16, 2025. https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/, 2025. Accessed 2026-08-02.

[53] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. Design patterns for securing LLM agents against prompt injections. arXiv:2506.08837, https://arxiv.org/abs/2506.08837, 2025.

[54] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629, https://arxiv.org/abs/2210.03629.

[55] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2302.04761, https://arxiv.org/abs/2302.04761.

[56] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2303.11366, https://arxiv.org/abs/2303.11366.

[57] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2303.17651, https://arxiv.org/abs/2303.17651.

[58] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798, https://arxiv.org/abs/2310.01798.

[59] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. arXiv:2304.05128, https://arxiv.org/abs/2304.05128, 2023.

[60] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model

calls into self-improving pipelines. arXiv:2310.03714, https://arxiv.org/abs/2310.03714, 2023.

[61] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, https://arxiv.org/abs/2310.08560, 2023.

[62] Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 2023. https://doi.org/10.1145/3586183.3606763.

[63] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. arXiv:2305.16291, https://arxiv.org/abs/2305.16291.

[64] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI-24)*, 2024. arXiv:2308.10144, https://arxiv.org/abs/2308.10144.

[65] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. arXiv:2409.07429, https://arxiv.org/abs/2409.07429, 2024.

[66] Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. CLIN: A continually learning language agent for rapid task adaptation and generalization. arXiv:2310.10134, https://arxiv.org/abs/2310.10134, 2023.

[67] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.00352, https://arxiv.org/abs/2308.00352.

[68] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2307.07924, https://arxiv.org/abs/2307.07924.

[69] Fake Lin, Binbin Hu, Xi Zhu, Ziwei Zhao, Zhi Zheng, Ziqi Liu, Zhiqiang Zhang, Jun Zhou, and Tong Xu. SmellBench: Towards fine-grained evaluation of code agents on refactoring tasks. arXiv:2606.05574, https://arxiv.org/abs/2606.05574, 2026.

[70] Alex Thillen, Niels Mündler, Veselin Raychev, and Martin Vechev. CodeTaste: Can LLMs generate human-level code refactorings? arXiv:2603.04177, https://arxiv.org/abs/2603.04177, 2026.

[71] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering (FSE)*, 2024. https://doi.org/10.1145/3660810.

[72] Anthropic. Introducing agent skills. Blog post, October 16, 2025. https://claude.com/blog/skills, 2025. Accessed 2026-08-02.

[73] OpenAI, Google, Cursor, Factory, and Sourcegraph. AGENTS.md: A simple, open format for guiding coding agents. https://agents.md/, 2025. Stewarded by the Agentic AI Foundation (Linux Foundation) since December 2025. Accessed 2026-08-02.

[74] Jialong Chen, Xander Xu, Hu Wei, Chuan Chen, and Bing Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. arXiv:2603.03823, https://arxiv.org/abs/2603.03823, 2026.

[75] Nenad Tomasev, Matija Franklin, Joel Z. Leibo, Julian Jacobs, William A. Cunningham, Iason Gabriel, and Simon Osindero. Virtual agent economies. arXiv:2509.10147, https://arxiv.org/abs/2509.10147, 2025.

[76] Jason Wu, Amanda Swearngin, Arun Krishna Vajjala, Alan Leung, Jeffrey Nichols, and Titus Barik. Improving user interface generation models from designer feedback. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '26)*, 2026. arXiv:2509.16779, https://arxiv.org/abs/2509.16779.

[77] Xuan Liu, Haoyang Shang, and Haojian Jin. Diagon: A programmable testbed for AI-agent cognitive labor markets. arXiv:2604.06688, https://arxiv.org/abs/2604.06688, 2026.